

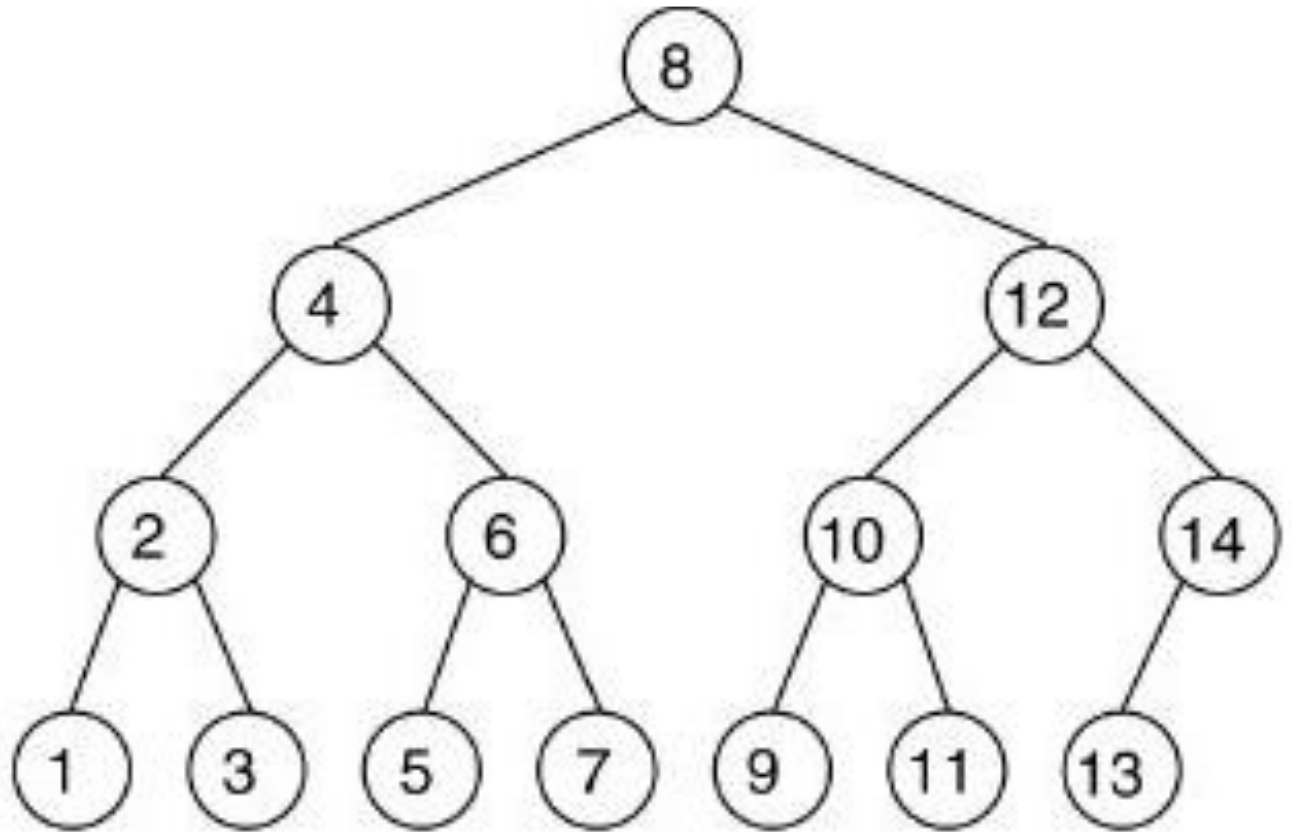
# Árvores Balanceadas

Estruturas de Dados e Algoritmos

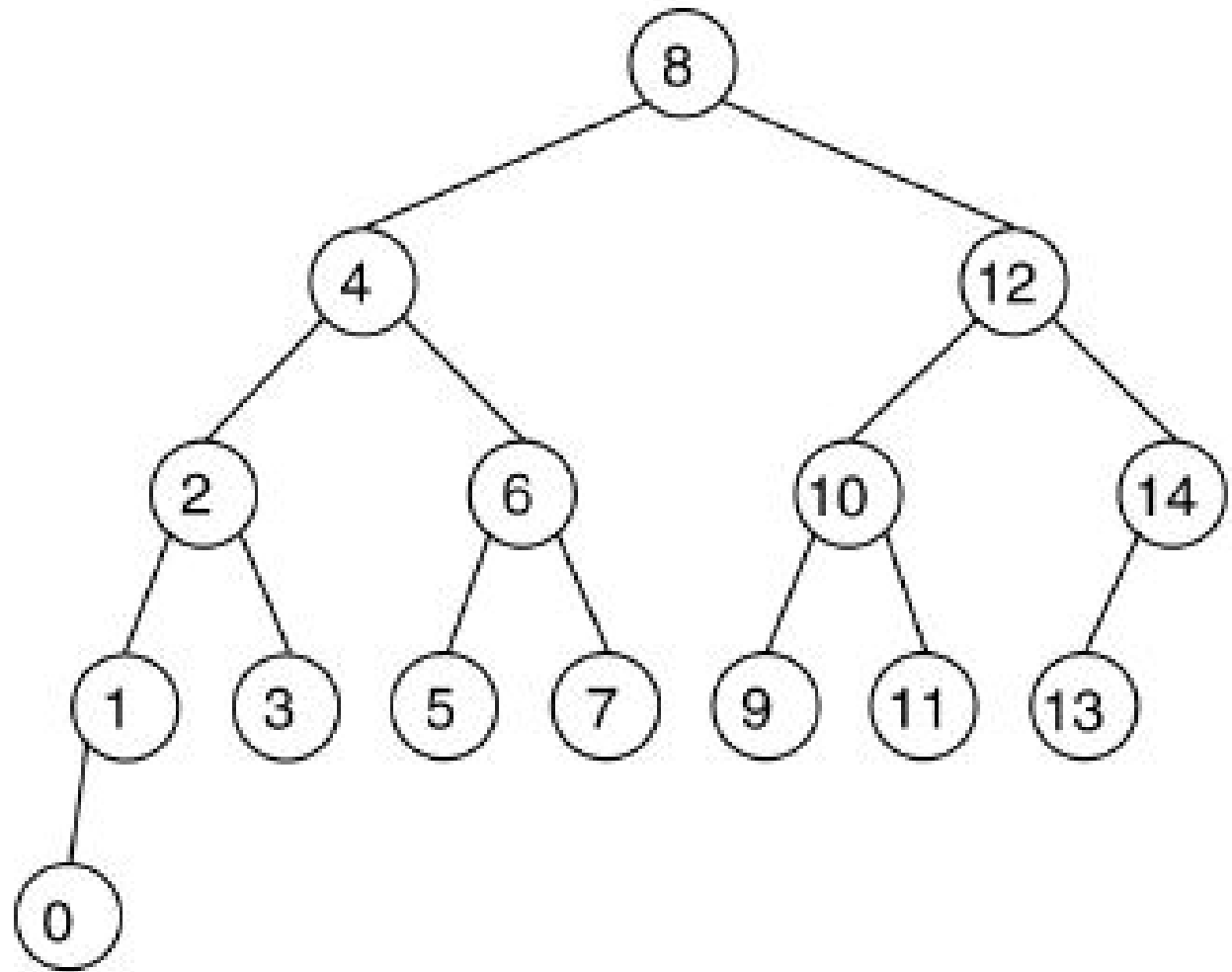
Prof. Sérgio Gorender

- Árvores binárias de busca possuem um custo de acesso de  $O(\log n)$ , quando são árvores binárias ótimas.
- Entretanto, após inserções e remoções, estas árvores tendem a não mais apresentarem esta característica.
- A solução é alterar periodicamente a estrutura da árvore, para que esta fique sempre próxima a uma árvore binária ótima.
- Chamamos estas árvores de balanceadas.
- Esta alteração periódica precisa ser de tal forma que seu custo computacional não seja alto.

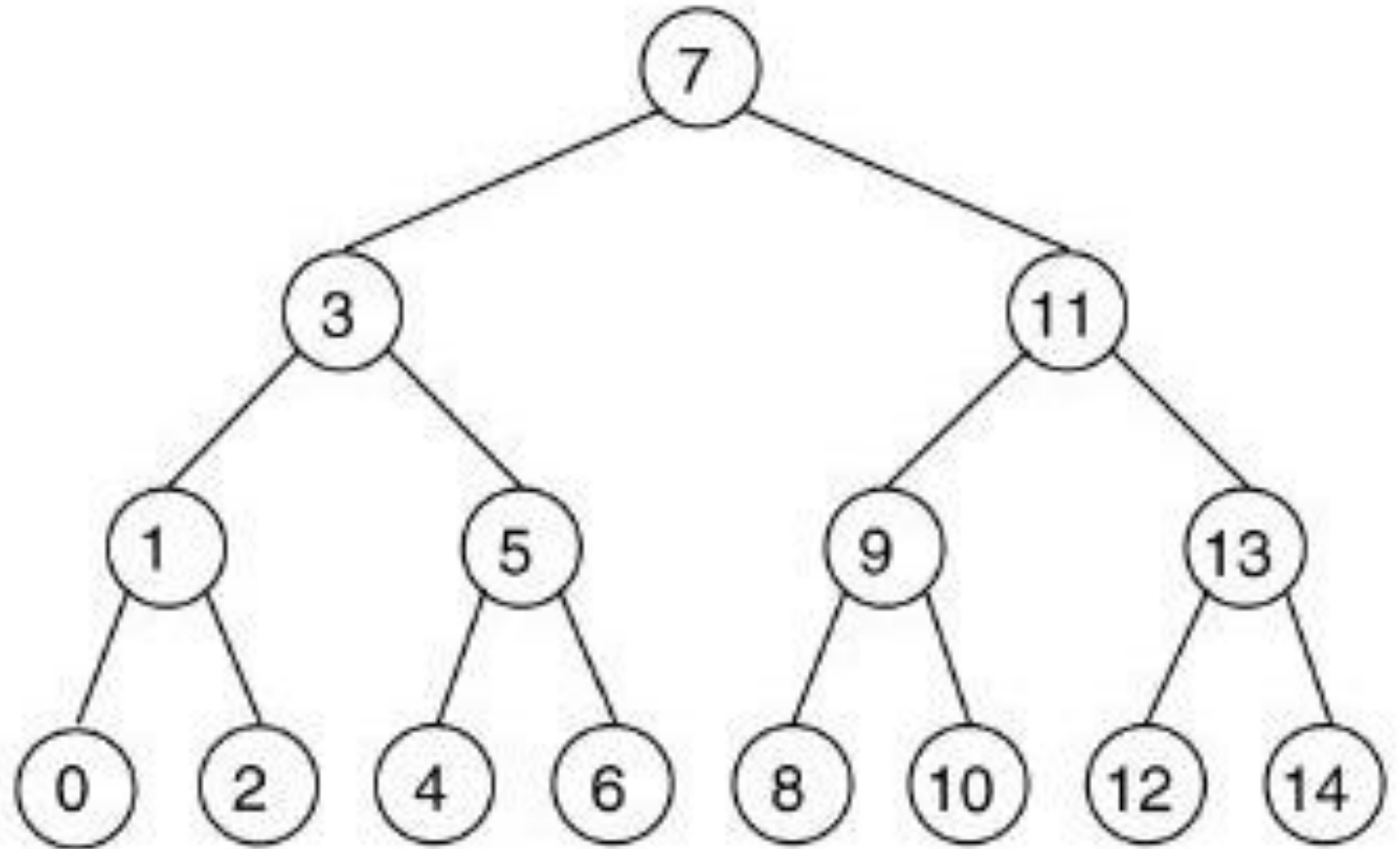
- Considere a árvore a seguir.
- Para ser mantida sua altura, o único nó que poderia ser inserido é um maior do que 14.



- Resultado da árvore caso seja inserido um nó inferior a todos os da árvore.
- Esta fica desbalanceada.



- Árvore resultante após alterar sua estrutura para ficar completa.
- Tal algoritmo requer ao menos  $\Omega(n)$  passos (percorre toda a árvore).



- Uma árvore balanceada não precisa ser completa. É definida como uma árvore na qual a altura seja igual a  $O(\log n)$ , para  $n$  nós, e que esta propriedade seja satisfeita por todas as suas subárvores.

# Árvore AVL

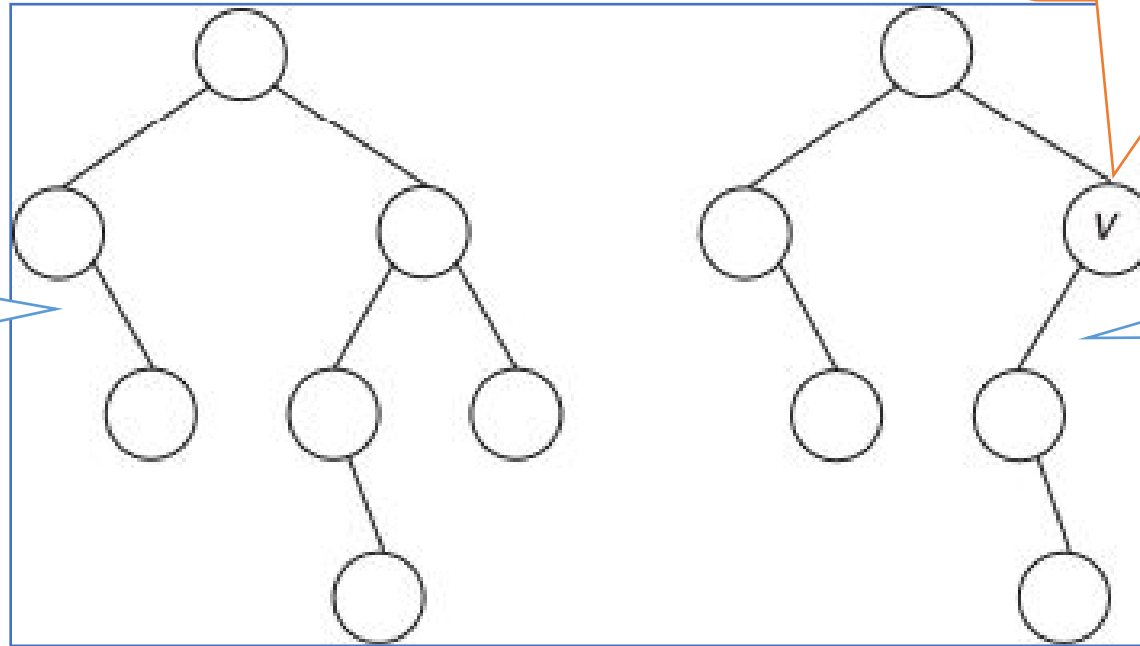
- Uma árvore binária **T** é denominada de árvore AVL, quando para qualquer nó de **T**, as alturas de suas duas subárvores esquerda e direita, diferem em módulo de no máximo **1**.
- Um nó que satisfaz esta propriedade é dito ser regulado. Um nó que não satisfaz a propriedade é dito desregulado, e a árvore que contém um nó assim também é dita desregulada.

- Árvore AVL

Nó desregulado

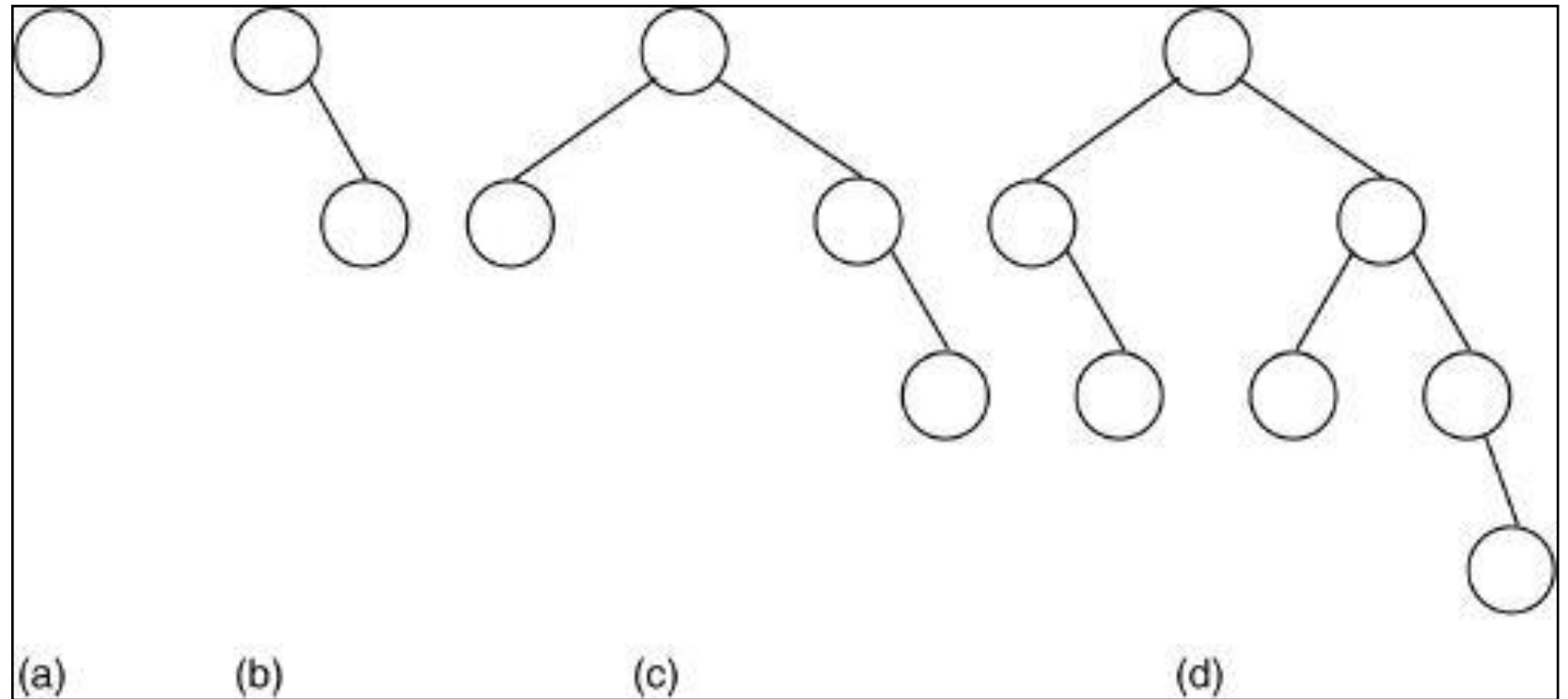
Árvore balanceada

Árvore não balanceada



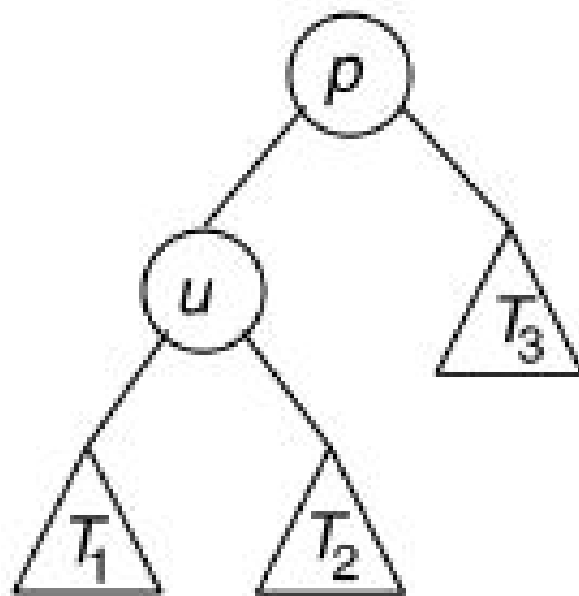


Exemplos de  
árvore AVL com  
alturas iguais a **1**,  
**2**, **3** e **4**.



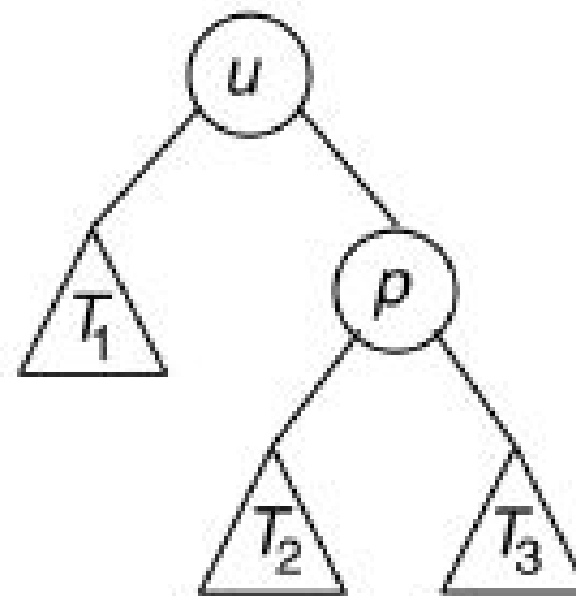
- Inclusão em árvore AVL
- A cada inclusão é preciso verificar se algum nó ficou desregulado – a diferença da altura de suas subárvores direita e esquerda for maior do que 1.
- São utilizadas 4 transformações possíveis:
  - Rotação direita, rotação esquerda, rotação dupla direita, rotação dupla esquerda.
- Se após uma inserção em uma árvore, algum nó ficar desregulado, este nó estará entre o que foi inserido e a raiz. Então basta percorrer a árvore entre o nó inserido e a raiz, testando cada nó com relação a ser desregulado.

(a)

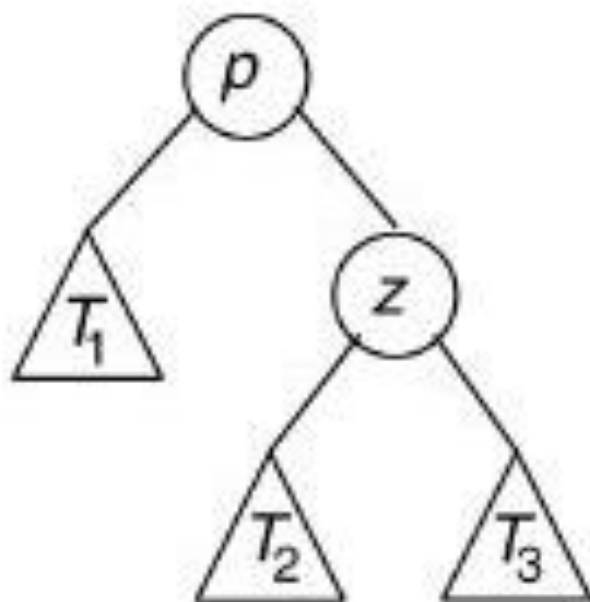


rotação  
direita

(b)

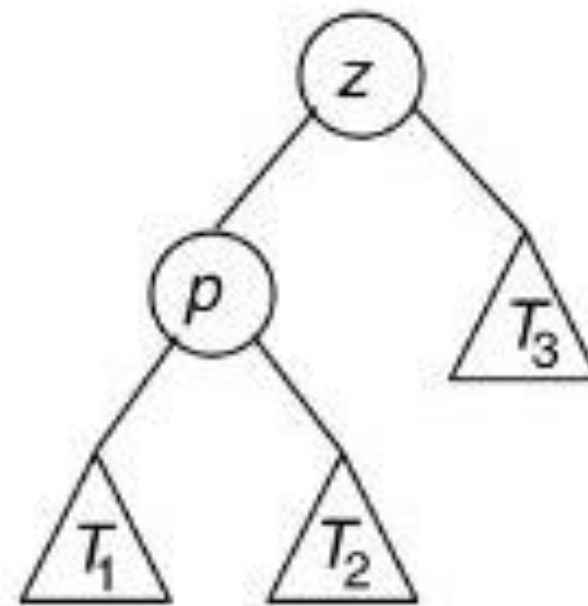


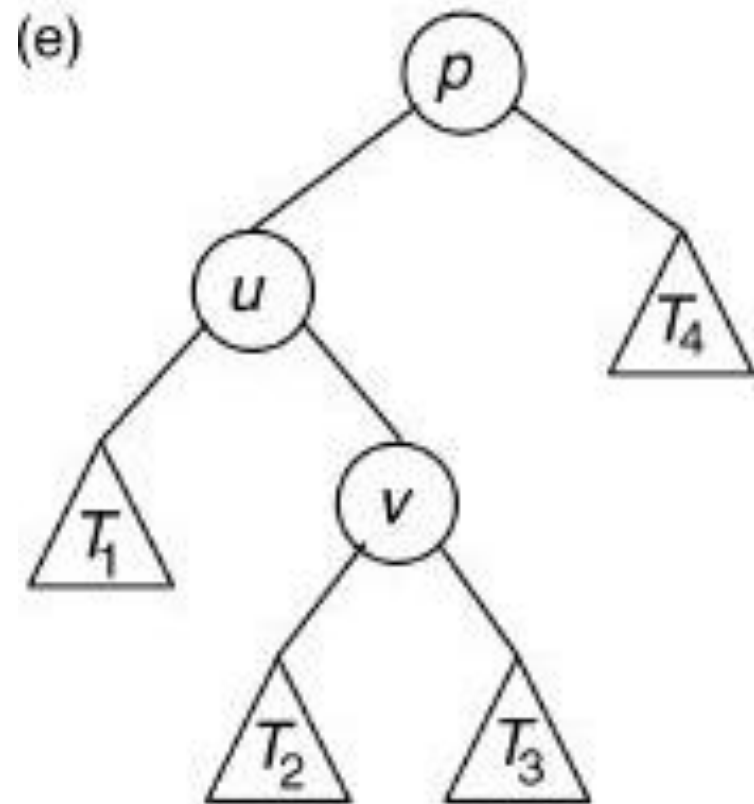
(c)

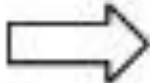


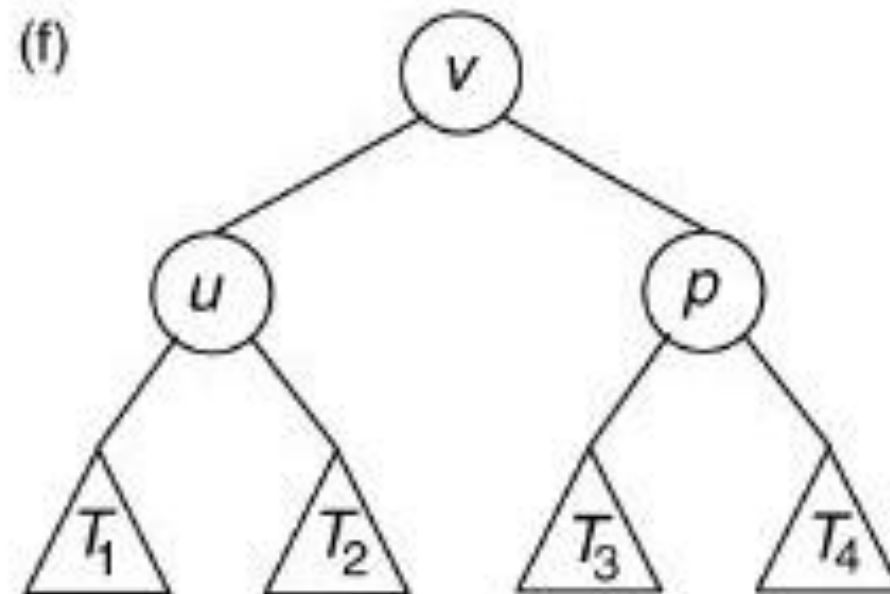
→  
rotação  
esquerda

(d)

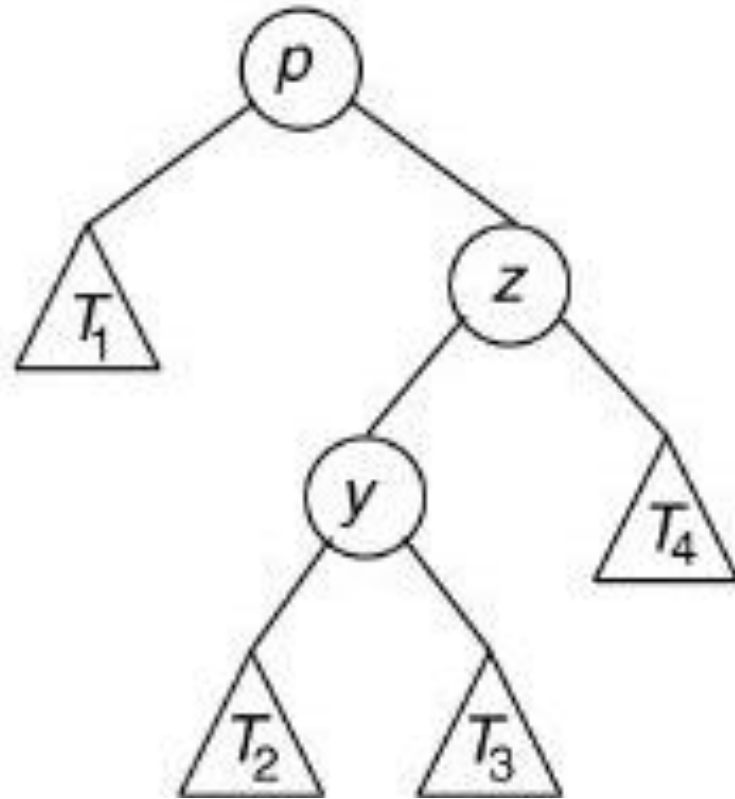




  
 rotação  
 dupla  
 direita

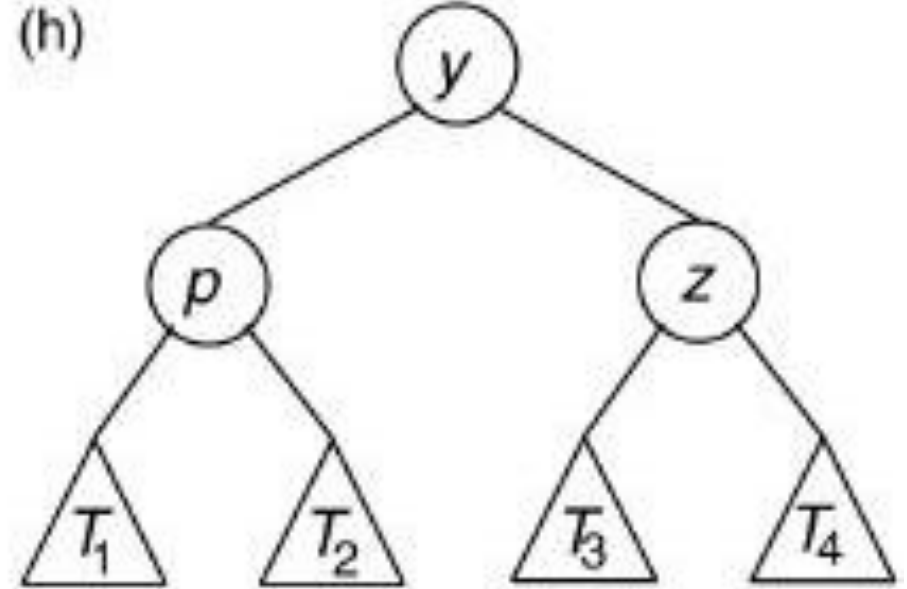


(g)



→  
rotação  
dupla  
esquerda

(h)



- Temos 4 situações possíveis que irão caracterizar cada uma das rotações.
- Vamos considerar **p** como o nó desregulado mais próximo às folhas, no caminho entre o nó inserido (uma folha) e a raiz da árvore, **u** como seu filho esquerdo e **z** como seu filho direito.

### **Caso 1:** $h_E(p) > h_D(p)$

O nó inserido,  $q$ , pertence à subárvore esquerda de  $p$ , sendo diferente de  $u$  (se  $q$  fosse igual a  $u$  o nó  $p$  não estaria desbalanceado). Temos que  $h_E(u) \neq h_D(u)$ . Temos então:

#### **Caso 1.1:** $h_E(u) > h_D(u)$

Neste caso deve ser feita uma rotação à direita, regulando o nó  $p$  e a árvore.

#### **Caso 1.2:** $h_D(u) > h_E(u)$

O nó  $u$  possui um filho direito  $v$ . Neste caso deve ser aplicada uma rotação dupla à direita da raiz  $p$ , regulando a árvore.



## Caso 2: $h_D(p) > h_E(p)$

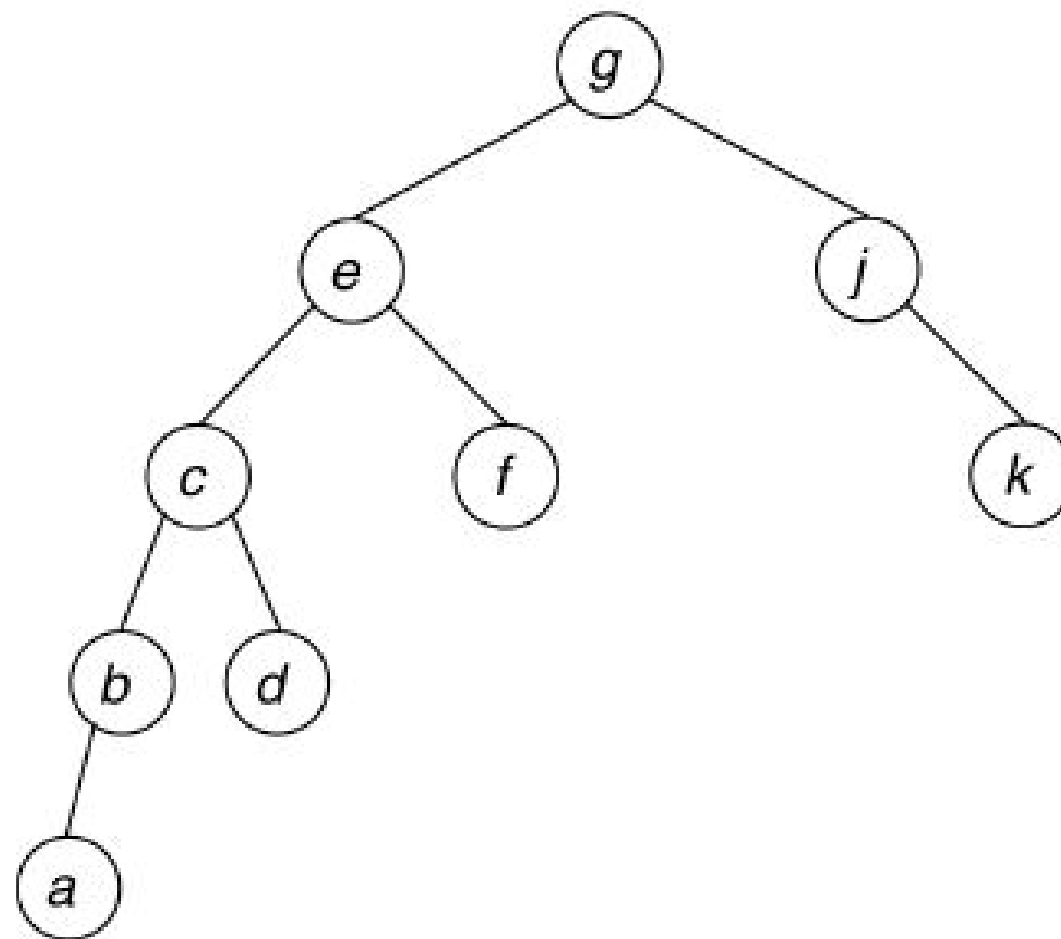
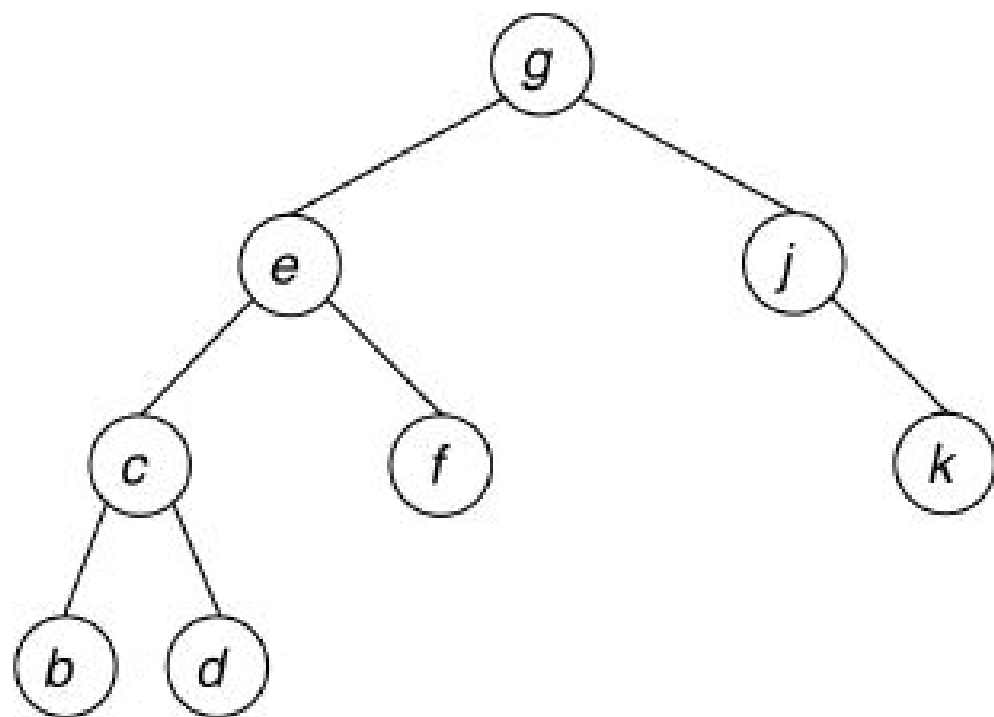
O nó inserido,  $q$ , pertence à subárvore direita de  $p$ , sendo diferente de  $z$  (se  $q$  fosse igual a  $z$  o nó  $p$  não estaria desbalanceado). Temos que  $h_E(z) \neq h_D(z)$ . Temos então:

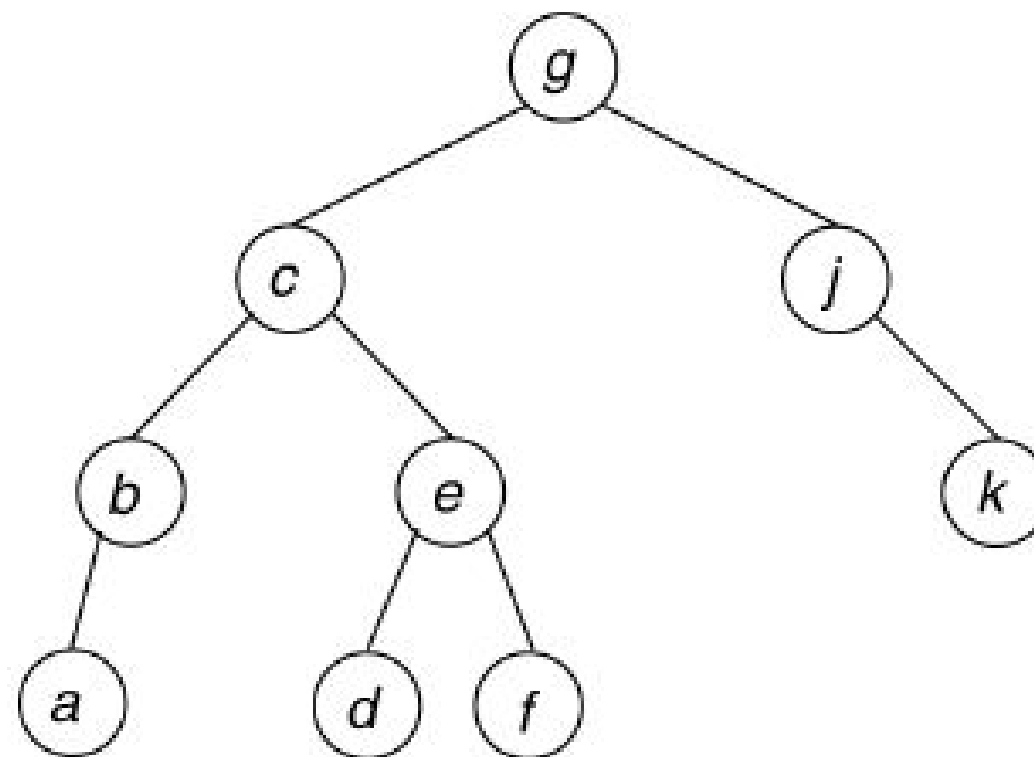
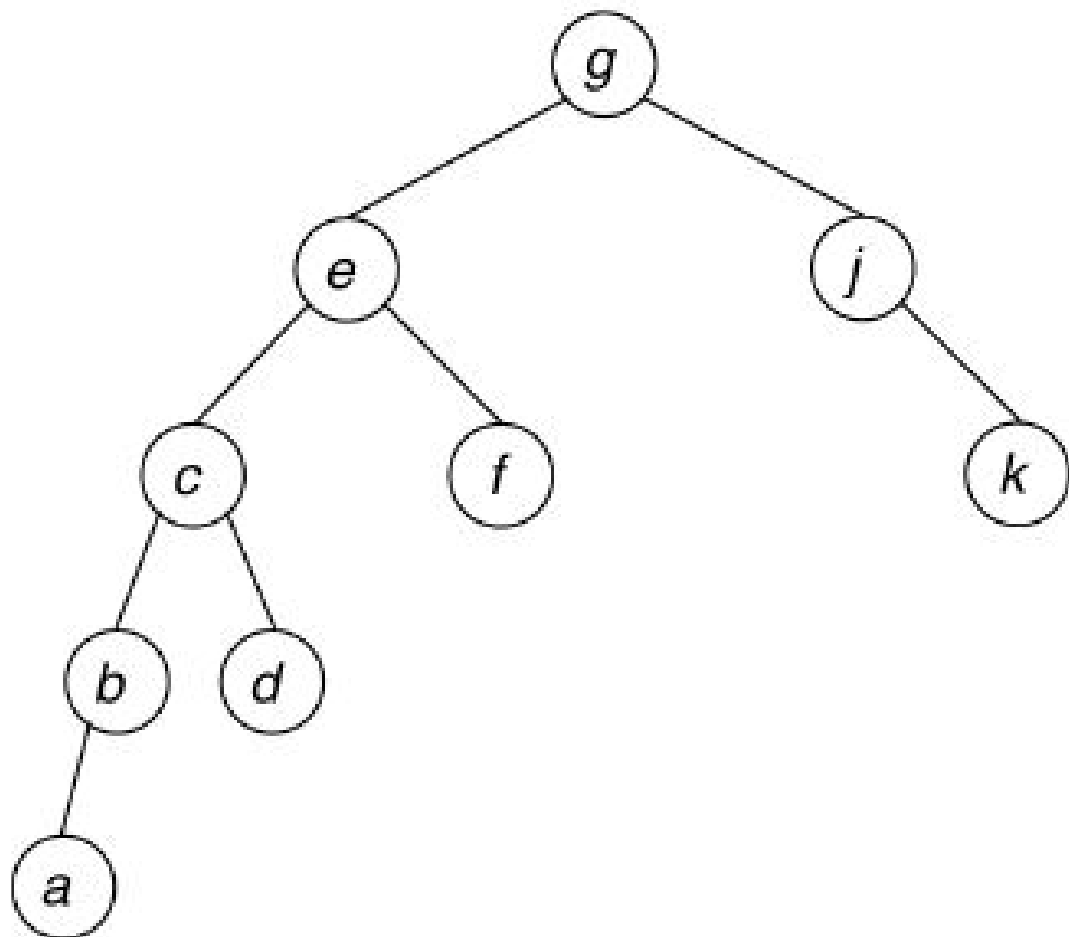
### Caso 2.1: $h_D(z) > h_E(z)$

Neste caso deve ser feita uma rotação à esquerda, regulando o nó  $p$  e a árvore.

### Caso 2.2: $h_E(z) > h_D(z)$

O nó  $z$  possui um filho direito  $y$ . Neste caso deve ser aplicada uma rotação dupla à esquerda da raiz  $p$ , regulando a árvore.





# Algoritmo de Inclusão em Árvore AVL

```
function inicio-no(pt)
    new(pt)
    pt.esq = Nulo;
    pt.dir = Nulo
    pt.chave = x;
    pt.bal = 0
```

```
function rot-dir(pt,h)
    ptu = pt.esq
    if ptu.bal == -1 then    #rotação simples
        pt-.esq = ptu.dir; ptu.dir = pt
        pt.bal = 0; pt = ptu
    else    ptv = ptu.dir #rotação dupla
        ptu.dir = ptv.esq; ptv.esq = ptu
        pt.esq = ptv.dir; ptv.dir = pt
        if ptv.bal == -1 then pt.bal = 1 else pt.bal = 0
        if ptv.bal == 1 then ptu.bal = -1 else ptu.bal = 0
        pt = ptv
    pt.bal = 0; h = f
    return pt, h
```

```
function rot-esq(pt,h)
    ptu = pt.dir
    if ptu.bal == 1 then
        pt.dir = ptu.esq; ptu.esq = pt
        pt.bal = 0; pt = ptu
    else ptv = ptu.esq
        ptu.esq = ptv.dir; ptv.dir = ptu
        pt.dir = ptv.esq; ptv.esq = pt
        if ptv.bal == 1 então pt.bal = -1 then pt.bal = 0
        if ptv.bal == -1 então ptu.bal = 1 then ptu.bal = 0
        pt = ptv
    pt.bal = 0; h = F
    return pt, h
```

```

function ins-AVL(x, pt, h)
    if pt == Nulo then
        new(pt)
        h = V
    else
        if x < pt.chave then
            pt.esq, h = ins-AVL(x, pt.esq, h)
            if h then
                if pt.bal == 1 then
                    pt.bal = 0; h = F
                else if pt.bal == 0 then
                    pt.bal = -1
                else if pt.bal == -1 then
                    pt, h = rot-dir(pt, h)
            else if pt.chave < x then
                pt.dir, h = ins-AVL(x, pt.dir, h)
                if h then
                    if pt.bal == -1 then
                        pt.bal = 0; h = F
                    else if pt.bal == 0 then
                        pt.bal = 1
                    else if pt.bal == 1 then
                        pt, h = rot-esq(pt, h)
        return pt, h

```



# Algoritmo de Exclusão em Árvore AVL

**procedimento** balanceEsq(pt, h)

**caso** p->bal

-1: pt.bal = 0

0: pt.bal = 1; h=FALSE

1:       p1 = pt.dir

         b1 = p1.bal

**se** b1 >= 0 **então** (\* rotação simples \*)

         p.dir = p1.esq

         p1.esq = p

**se** b1 == 0 **então**

         p.bal = 1

         p1.bal = -1

         h = FALSE

**senão**

         p.bal = 0

         p1.bal = 0

     p = p1

```
senão (* rotação dupla *)  
    p2 = p1.esq  
    b2 = p2.bal  
    p1.esq = p2.dir  
    p2.dir = p1  
    p1.dir = p2.esq  
    p2.esq = p  
    se b2 == +1 então  
        p.bal = -1  
    senão  
        p.bal = 0  
    se b2 == -1 então  
        p1.bal = 1  
    senão  
        p1.bal = 0  
    p = p2  
    p2.bal = 0
```

**procedimento** balanceDir(pt,h)

**caso** pt.bal

1 : pt.bal = 0

0 : pt.bal = -1 ; h = FALSE

-1 : p1 = p.esq

b1 = p1.bal

**se** b1 <= **então** (\* rotação simples \*)

p.esq = p1.dir

p1.dir = p

**se** b1 === 0 **então**

p.bal = -1

p1.bal = -1

h = FALSE

**senão**

p.bal = 0

p1.bal = 0

p = p1

```
senão (* rotação dupla *)  
    p2 = pt.dir  
    b2 = p2.bal  
    p1.dir = p2.esq  
    p2.esq = p1  
    pt.esq = p2.dir  
    p2.dir = pt  
    se b2 == -1 então  
        pt.bal = 1  
    senão  
        p.bal = 0  
    se b2 == 1 então  
        p1.bal = -1  
    senão  
        p1.bal = 0  
    p = p2  
    p2.bal = 0
```

```
procedimento exc(r, h)
se r.dir != NUL então
    exc(r->dir, h)
    se h então
        balanceD(r, h)
senão
    q->chave = r->chave
    q->cont = r->cont
    q = r
    r = r->esq
    h = TRUE
```

```

procedimento excluir(x, p, h)
se p == Nulo então                                (a chave não se encontra na árvore)
senão
    se p.chave > x então
        excluir(x, p.esq, h)
        se h então
            balanceEsq(p, h)
    senão
        se p.chave < x então
            excluir(x, p.dir, h)
            se h então
                balanceDir(p, h)
        senão
            q = p
            se q.dir == Nulo então
                p = q.esq
                h = TRUE
            senão
                se q.esq == Nulo então
                    p = q.dir
                    h = TRUE
                senão
                    exc(q.esq, h)
                    se h então
                        balanceEsq(p, h)
            desalocar(q)

```